AR-Based Wall & Floor Color Visualization with

ML-Based Color Recommendation

(Analysis and Design- Final Year Project)

ITBNM-2211-0184 - J.M.V.SANDARU

ITBIN-2211-0307 – M.T.UPEKSHA

Contents

CHAPTER 1 3
INTRODUCTION 3
CHAPTER 2 3
LITERATURE REVIEW 3
CHAPTER 3 4
ANALYSIS 4
3.1 Introduction 4
3.2 Feasibility Study 4
3.3 Fact Finding Techniques 5
3.4 Requirement Analysis 5
3.5 Functional Requirements 6
3.6 Non-Functional Requirements 6
3.7 Methodology for the System Development 6
CHAPTER 4 7
DESIGN 7
4.1 Introduction 7
4.2 System Design Process 7
4.3 Interface Design 12
4.4 Database Design 15
4.5 Hardware Component (If Any) 16
Chapter 4 16
Conclusion 16

CHAPTER 1

INTRODUCTION

Interior design plays a crucial role in shaping the visual appeal, functionality, and comfort of indoor spaces. Among the various design elements, selecting suitable wall and floor colors is one of the most influential yet challenging tasks. Traditional decision-making methods, such as using physical color samples, printed catalogues, or 2D visualizations, often fail to provide an accurate representation of how colors appear in real environments under dynamic lighting conditions. As a result, individuals frequently struggle to make confident color choices, leading to dissatisfaction, increased costs, and unnecessary design revisions.

With advancements in technology, Augmented Reality (AR) has emerged as a powerful tool capable of bridging the gap between imagination and reality by enabling users to visualize virtual colors and textures within their actual environments. Meanwhile, Artificial Intelligence (AI), particularly Machine Learning (ML), has proven to be highly effective in understanding user preferences and offering personalized suggestions. Although both technologies have shown great potential, existing interior design tools typically treat visualization and personalized recommendation as separate functions, limiting the overall decision-making experience.

This research project proposes the development of a mobile application that integrates AR-based color visualization with ML-driven color palette recommendation to support users in making informed interior design decisions. The chapter provides the background of the study, the research problem, the significance and motivation for the project, as well as the aims, objectives, limitations, and the chapter layout of the report.

CHAPTER 2

LITERATURE REVIEW

Interior design research has increasingly explored the use of immersive technologies and intelligent algorithms to enhance visualization, user engagement, and decision-making. Traditional interior design techniques—such as physical color samples, sketches, and design boards—are limited by subjective interpretation and poor scalability. Researchers have therefore turned toward AR for real-time spatial visualization and ML for intelligent, data-driven recommendations.

However, despite individual advancements in both domains, literature suggests that **most solutions treat AR and ML as separate components**, creating a gap that your project aims to address. This chapter provides a structured interpretation of these research areas and demonstrates how their convergence offers novel opportunities for improving interior design workflows.

CHAPTER 3

ANALYSIS

3.1 Introduction

This chapter presents an in-depth analysis of the proposed Augmented Reality–based interior design mobile application with personalized color palette recommendation. The analysis includes a feasibility study, identification of system requirements, functional and non-functional specifications, user characteristics, and the prerequisites for system use. A top-level use case diagram is also described to illustrate user interactions with the system. The chapter concludes by explaining the development methodology selected for the project.

3.2 Feasibility Study

The feasibility study evaluates whether the proposed system can be successfully developed and deployed within the available constraints of time, technology, skills, and resources.

3.2.1 Technical Feasibility

The system can be implemented with currently available and accessible technologies:

- **ARCore SDK** supports real-time AR tracking, surface detection, and color overlay.
- **Unity** provides cross-platform development with a single codebase.
- **Python ML libraries** (TensorFlow / scikit-learn) enable efficient implementation of recommendation models.
- **Firebase** facilitates real-time data storage, authentication, and cloud hosting.

3.2.2 Operational Feasibility

The system provides the following operational benefits:

- Simplifies wall and floor color decision-making.
- Offers personalized recommendations based on user preferences.
- Requires no professional interior design experience.
- Provides immediate feedback through AR visualization.

3.2.3 Economic Feasibility

Most tools used are free or open-source:

- Unity – Free
- Android Studio – Free
- ARCore – Free
- Firebase – Free tier
- Python ML libraries – Free

Costs are minimal and primarily involve developer time.

3.2.4 Schedule Feasibility

- The project can be completed within the academic time period.

- The workload is divided into modules

Each module can be completed step by step.

3.3 Fact Finding Techniques

Several techniques were used to gather accurate and user-centered requirements.

3.3.1 Literature Review
Used to understand research gaps, technologies, existing systems, and user pain points.
3.3.2 User Surveys
Conducted with potential app users (homeowners, students, small interior designers).
Collected insights on color selection difficulties, expectations from AR, and interest in personalized features.
3.3.3 Observations
Analyzed how users typically choose wall/floor colors at paint stores or through online catalogues.
3.3.4 Comparative Analysis
Studied existing applications such as IKEA Place and Houzz to identify strengths and limitations.

3.4 Requirement Analysis

3.4.1 User Characteristics

User Type	Description	Frequency of Use	Skill Level
Homeowners / General Users	Want to visualize wall and floor colors	Occasional	Beginner
Interior Designers	Use app to demonstrate ideas to clients	Frequent	Intermediate
Students / Learners	Use for visualization and color harmony understanding	Occasional	Beginner
Administrators	Maintain ML models and manage backend datasets	Rare	Advanced

3.4.2 System Scope

The system will:

- Allow users to visualize colors on walls and floors in real time using AR.
- Generate personalized color palette recommendations using ML.
- Store user preferences in a secure database.
- Allow users to save and compare color variations.

The system will not include:

- Full furniture placement features
- Auto-room measurement

- Advanced lighting simulation

3.5 Functional Requirements

The system provides a range of functional capabilities designed to support users in selecting and visualizing interior color options effectively. First, the application enables user registration and login, allowing individuals to create accounts and securely access the system through Firebase authentication. The core functionality includes AR-based color visualization, where ARCore detects walls and floors and overlays selected colors in real time, giving users the ability to switch between colors and save their AR previews. The system also integrates a personalized color recommendation module powered by a machine learning model that analyzes user preferences to generate aesthetically harmonious palettes across themes such as modern, warm, and minimalistic. Additionally, the color selection module allows users to browse predefined color sets or upload reference images to extract color palettes. To enhance decision-making, users can save their designs and compare multiple color selections or AR previews side-by-side. The system further includes feedback collection, enabling users to rate recommendations to help improve the ML model over time. Finally, an administrative module supports the management of datasets, allowing administrators to upload or update color libraries and maintain both the ML model and its training data..

3.6 Non-Functional Requirements

The system is designed to meet several important non-functional requirements to ensure smooth and reliable operation. In terms of performance, the AR visualization must load within 1–2 seconds, color overlays should respond in under 100 milliseconds, and machine learning recommendations are expected to generate results within 3 seconds. The application emphasizes usability, offering a user-friendly interface suitable for non-technical users, requiring minimal steps to visualize colors, and providing clear icons and guidance throughout. To ensure reliability, the AR tracking must remain stable under normal lighting conditions, and the system should maintain an uptime of at least 90% during user interactions. Regarding portability, the application must run on ARCore-compatible Android devices, with future potential to extend support to iOS using ARKit. The system follows strict security measures by storing user data securely in Firebase, encrypting authentication credentials, and limiting data collection to basic user preferences only. It also supports scalability through its cloud-based backend, allowing expansion to accommodate more users and additional color datasets. Finally, maintainability is ensured through a modular Flutter-based architecture that simplifies updates, and the separation of ML model training enables continuous improvement of recommendations without affecting the app's user interface.

3.7 Methodology for the System Development

Selected Methodology: Agile Development Methodology

The **Agile methodology** is selected due to its suitability for iterative, user-centered, and flexible system development, especially where interactions and real-time features like AR must be repeatedly tested.

Reasons for Selecting Agile

- Iterative development: AR and ML modules require multiple rounds of refinement.

- User feedback integration: Early prototypes can be tested with real users.
- Flexibility: Allows changes in design decisions or model improvement.
- Parallel development: AR and ML components can be built simultaneously.
- Reduced risk: Early detection of problems through sprint-based testing.

Agile Phases Applied in This Project

1. Requirement Gathering & Planning
2. Design Sprint (AR and ML module planning)
3. Prototype Development
4. User Testing & Feedback
5. Improvement Cycles
6. Final Integration
7. Deployment & Evaluation

CHAPTER 4

DESIGN

4.1 Introduction

This chapter presents the design of the proposed Augmented Reality (AR)–based interior design mobile application with Machine Learning (ML)–driven color palette recommendation. It begins by comparing alternative design strategies and selecting the most suitable one based on feasibility, cost, and alignment with system requirements.
The chapter then details the design of the system using standard modeling techniques including use case diagrams, class diagrams, activity diagrams, and sequence diagrams. Interface design principles, sample screen layouts, and database structures are also described. Together, these components provide a complete architectural understanding of the system.

4.2 System Design Process

The system design process consists of two major phases:

1. **Conceptual Design** – Identifying system components, interactions, workflows, and data structures.
2. **Technical Design** – Selecting technologies, frameworks, and architecture to implement the system.

4.2.1 Alternative Design Strategies

Three competing design approaches were considered:

Alternative 1: Full Custom Development (Built from Scratch)

Factor	Description
Platform	Flutter Framework (cross-platform)
AR Module	ARCore directly implemented in Flutter
ML Component	Custom ML model built with Python/TensorFlow
Backend	Firebase Cloud Firestore
Cost	Low (all tools free)
Pros	Maximum control, customizable ML model, scalable architecture
Cons	Higher development time, requires ML training expertise

Requirement Satisfaction
Custom AR overlays
Personalized ML recommendations
Real-time data storage
High maintainability

Alternative 2: Using Existing Open-Source AR + ML Libraries

Factor	Description
AR Module	Prebuilt open-source AR libraries (Vuforia, AR.js)
ML Component	Open-source recommendation libraries
Platform	Web/Hybrid apps
Pros	Faster development time
Cons	Limited customization, licensing issues, lower performance for mobile AR

Requirement Satisfaction
Limited real-time AR accuracy
Cannot perform custom ML recommendation logic
Faster initial development

Alternative 3: Native Android Development (Java/Kotlin + ARCore)

Factor	Description
Platform	Android SDK
AR Module	Native ARCore

Factor	Description
ML Component	TensorFlow Lite on-device model
Pros	Best AR performance, deep hardware access
Cons	Android-only, increased development complexity

Requirement Satisfaction
High AR performance
No cross-platform support
Greater complexity

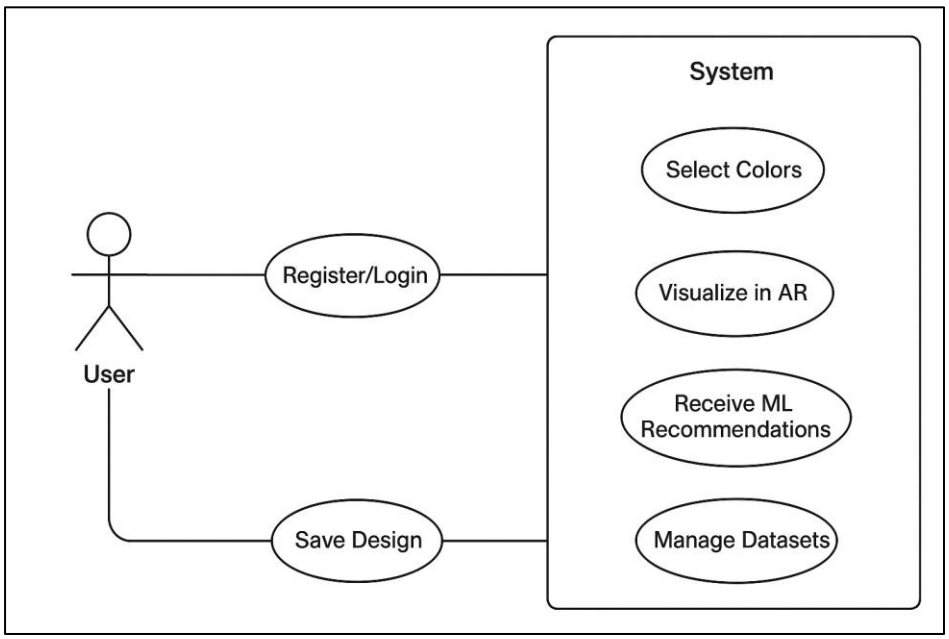
Selected Design Strategy: Full Custom Development (Alternative 1)

Justification:
This approach allows:

- Full control over AR visualization.
- A flexible ML recommendation engine that can evolve.
- Cross-platform potential (Android now, iOS later).
- Cost-effective development using open-source tools.
- Easy maintenance and future scalability.

Therefore, **Alternative 1** is selected as the final design strategy.

4.2.1 Use Case Diagram



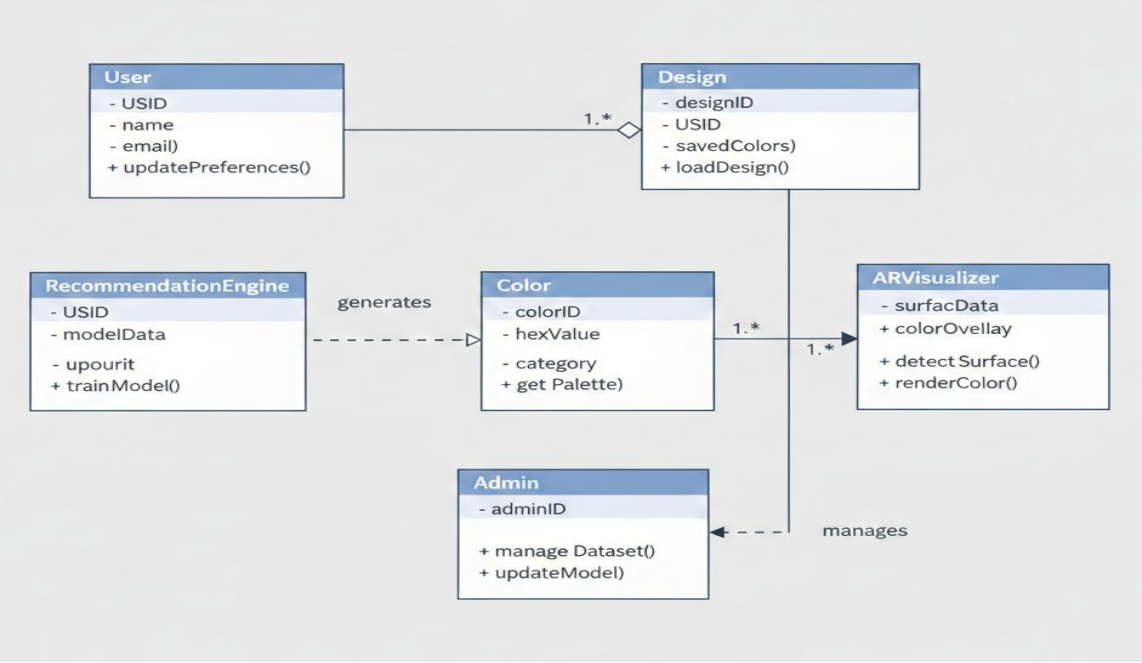
Description:
The use case diagram defines the interaction between actors (User, Administrator) and the system functions.
Main use cases include:

- Register/Login
- Select Colors
- Visualize in AR
- Receive ML Recommendations
- Save Design
- Manage Datasets (Admin)

4.2.2 Class Diagram

Main Classes

Class	Attributes	Methods
User	userID, name, email, preferences	login(), updatePreferences()
Color	colorID, hexValue, category	selectColor(), getPalette()
RecommendationEngine	modelData, userInput	generatePalette(), trainModel()
ARVisualizer	surfaceData, colorOverlay	detectSurface(), renderColor()
Design	designID, userID, savedColors	saveDesign(), loadDesign()
Admin	adminID	manageDataset(), updateModel()

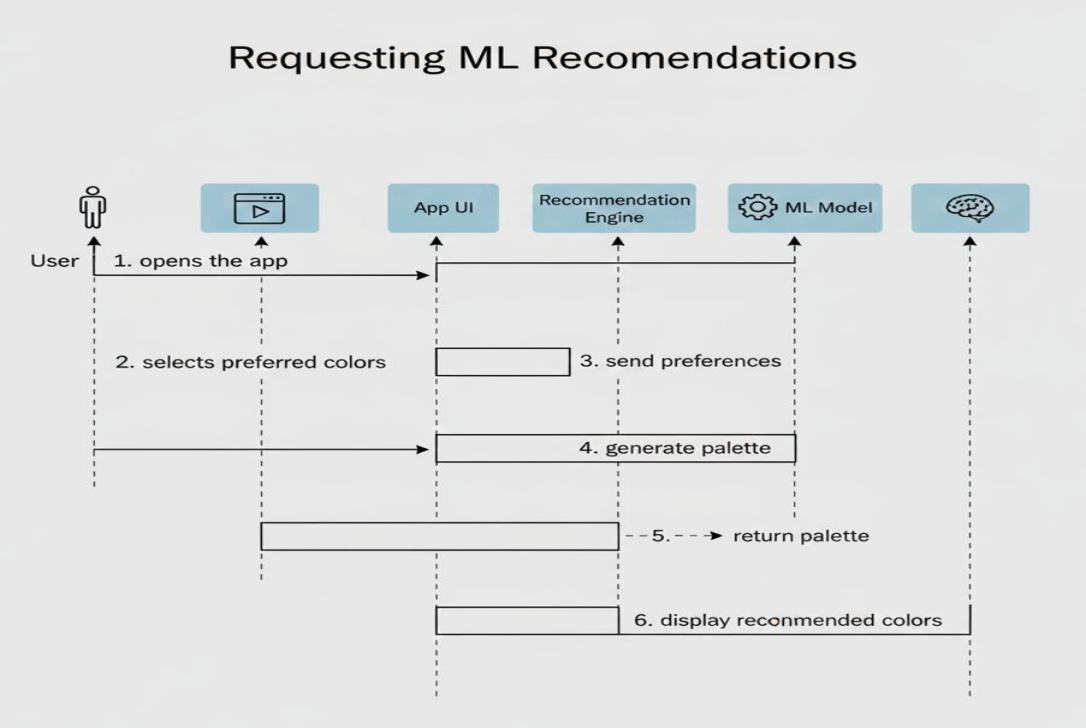


4.2.3 Sequence Diagrams

Sequence: Requesting ML Recommendations

1. User opens the app.
2. User selects preferred colors.
3. System sends preferences to Recommendation Engine.
4. ML model generates palette.
5. System displays recommended colors.

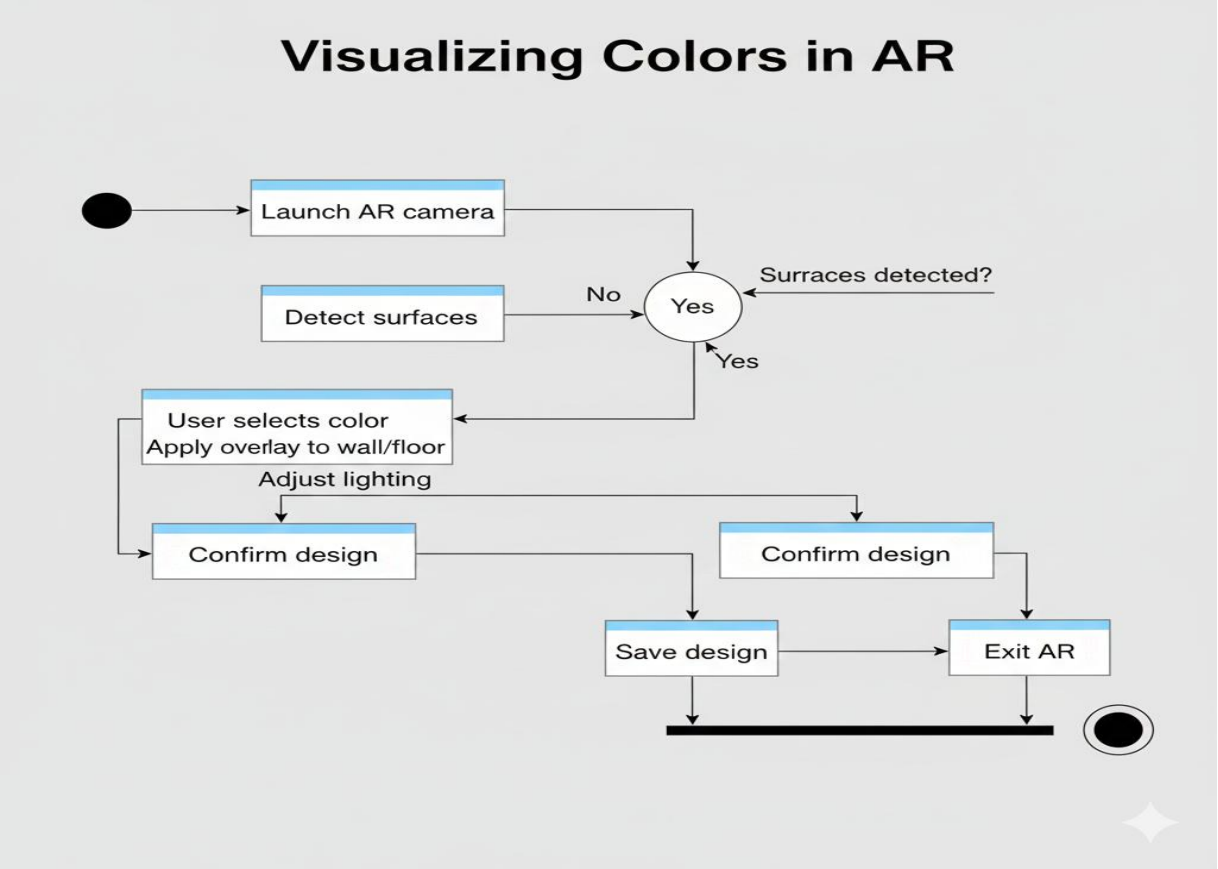
Another diagram can be created for AR visualization sequence.



4.2.4 Activity Diagrams

Activity: Visualizing Colors in AR

1. Launch AR camera
2. Detect surfaces
3. User selects color
4. Apply overlay to wall/floor
5. Adjust lighting
6. Confirm design
7. Save/Exit



4.3 Interface Design

4.3.1 Design Principles

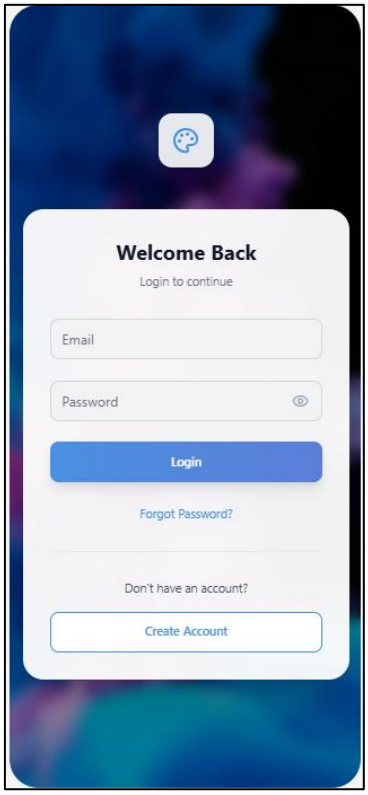
The interface follows:

- **Simplicity** – Users should complete tasks in minimal steps.
- **Consistency** – Same layout style across screens.
- **Responsiveness** – Suitable for different screen sizes.
- **Minimal Color Distraction** – Neutral UI to emphasize wall/floor colors.
- **Efficiency** – Key actions accessible within one or two taps.

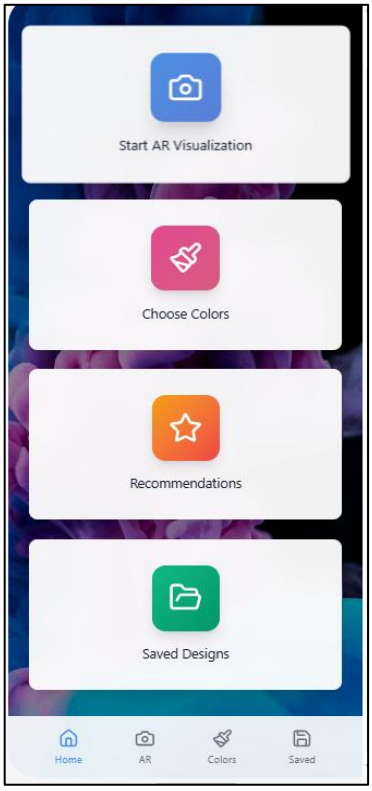
4.3.2 Actual Screens of the System

Screens include:

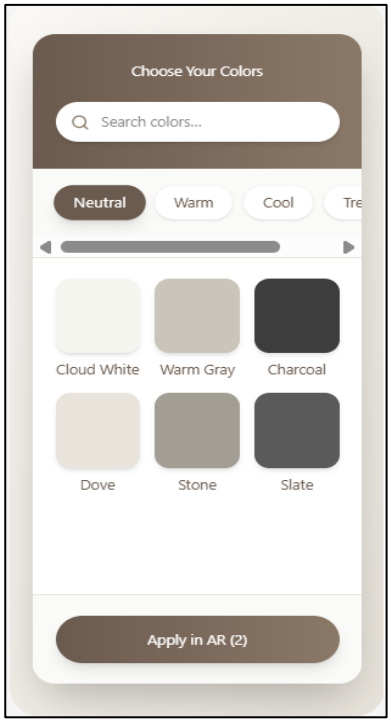
Login/Signup Screen



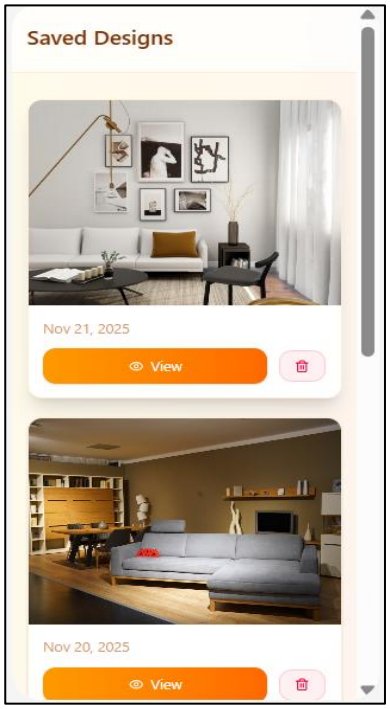
Home Dashboard



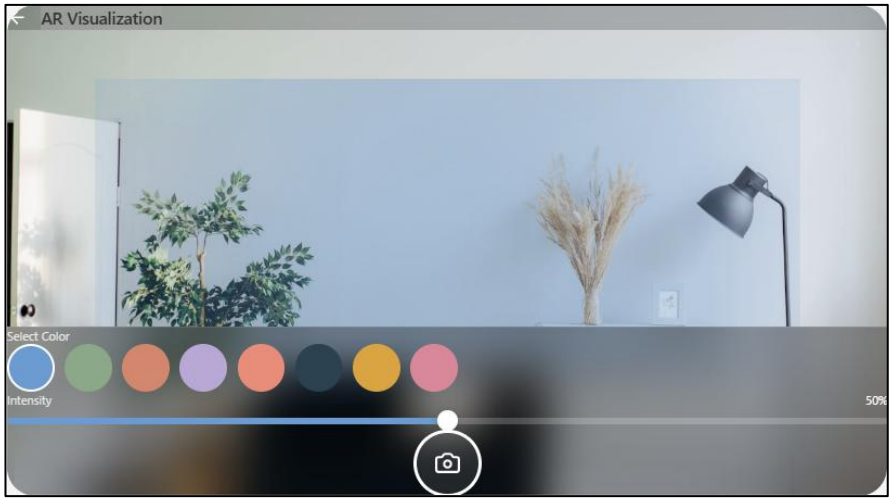
Color Selection Screen



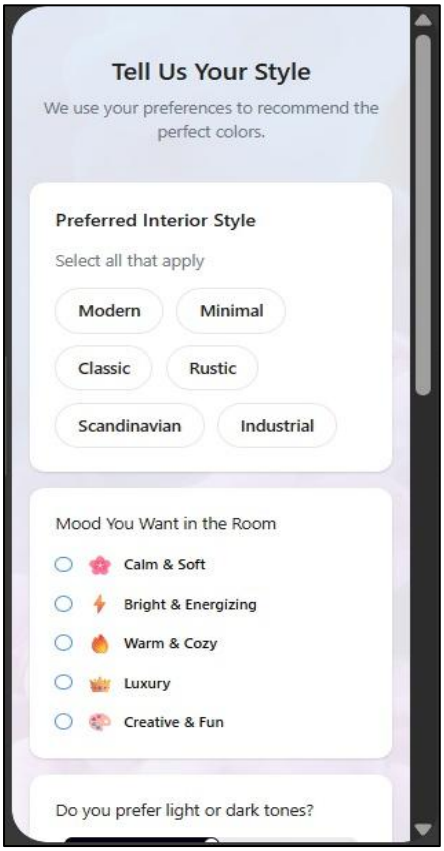
Saved Designs Screen

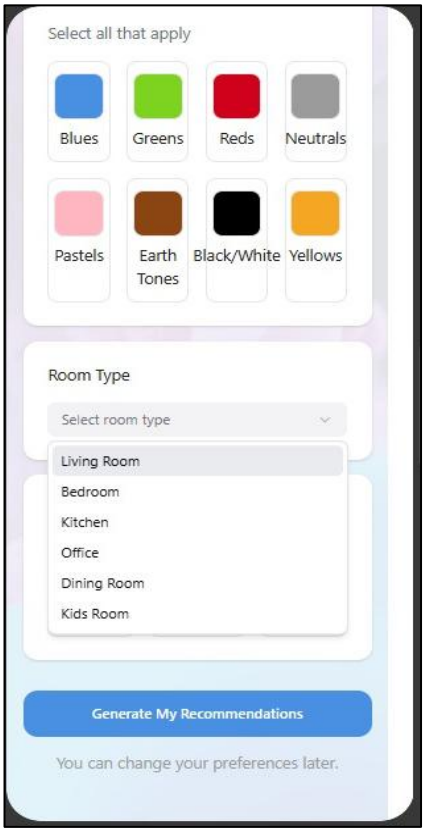


AR Visualization Screen



Recommendations Screen





4.4 Database Design

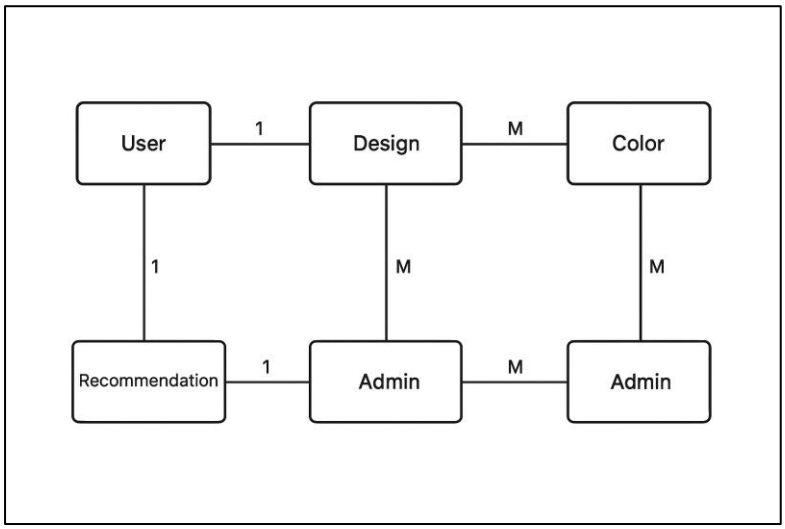
4.4.1 Entity Relationship Diagram (ERD)

Entities:

- User
- Design
- Color
- Recommendation
- Admin

Relationships:

- User (1) — (M) Design
- Design (M) — (M) Color
- Recommendation (M) — (1) User



4.4.2 Normalization

To ensure database efficiency:

1NF:

- All fields hold atomic values.

2NF:

- Composite keys removed.
- Color palette tables dependent on single keys.

3NF:

- No transitive dependencies.
- User preferences stored separately.

4.4.3 Relational Schema

User (userID, name, email, passwordHash, preferencesID)
Preferences (preferencesID, favoriteColors, styleType)
Design (designID, userID, dateSaved)
DesignColors (designID, colorID)
Color (colorID, hexValue, colorName, category)
Recommendation (recoID, userID, colors, timestamp)

4.5 Hardware Component (If Any)

This project primarily requires software, but minimum hardware specifications include:

User Device Requirements

- **Android Smartphone** with ARCore support
- Minimum 4GB RAM
- 64-bit processor
- Rear camera with autofocus
- Gyroscope + accelerometer for AR tracking

Development Hardware

- Laptop/PC
- Minimum 8GB RAM
- Flutter & Android Studio installed
- Python environment for ML model training

Chapter 4

Conclusion

The project successfully developed an **Augmented Reality (AR)-based mobile application** that integrates **real-time wall and floor color visualization with a personalized Machine Learning (ML) recommendation system**. This integrated solution successfully addressed the research gap of existing systems that treat visualization and recommendation separately.